Contents

CLAUDIU user manual

CLAUDIU (working codename) is a local, browser-based tabbed interface for running several Claude Code sessions at once. A small Python server spawns real `claude` CLI processes in pseudo-terminals and streams them into browser tabs via xterm.js, so every tab behaves exactly like a real terminal — same colors, same prompts, same keybindings — while the browser layer adds tabs, a calmer theme, adjustable fonts, snippets, and a project launcher on top.

Quick start

```
pip install -e .
python -m claudiu
```

or, on Windows, double-click `run_claudiu.bat`. The server starts on `http://127.0.0.1:8642/` (127.0.0.1 only — never reachable from other machines) and opens it in your default browser automatically.

Features

- **Tabs** — one tab per live `claude` session, named after the project folder and renamable.
- **Terminal fidelity** — each tab embeds a real interactive `claude` process rendered by xterm.js: skills, plugins, slash commands, and permission prompts all behave exactly as in a plain terminal.
- **Session survival on refresh** — sessions live on the server, not in the browser tab; refreshing or closing Chrome never kills them.
- **Reconnect with backoff** — a dropped websocket retries automatically with backoff behind a visible "reconnecting…" strip; it never fails silently.
- **Resume after crash** — after a browser or PC crash, a resume list offers one-click `claude --resume <session-id>` per recent project.
- **Snippets bar + palette** — frequently used prompt text as buttons under the tab strip and in a fuzzy-searchable palette (`Ctrl+k`).
- **Project launcher** — configured working folders (plus a browse option) in the new-session dialog; one click starts `claude` in the chosen directory.
- **Per-tab font size** — each tab's font size is adjustable and remembered independently.
- **Scrollback search** — search within a tab's capped scrollback, with highlighting.
- **Rename by double-click** — double-click a tab's label to rename it.
- **Calm dark theme** — near-black background, warm-gray foreground, muted ANSI colors, steady non-blinking cursor; every color is a config value.

Configuration

One human-editable JSON file. Default location `~/.claudiu/config.json`, created with defaults the first time the server runs if it does not exist yet. Override the path with the `CLAUDIU_CONFIG` environment variable, or with the `--config` flag on the command line. Keys the file does not mention fall back to the defaults below; keys

the file has that this version of CLAUDIU does not recognize are kept as-is and reported as a startup warning, never rejected.

Key	Default	Meaning
`port`	`8642`	TCP port the server listens on (bound to 127.0.0.1 only)
`claude_command`	`["claude"]`	argv used to spawn each session
`replay_chunks`	`2000`	max buffered output chunks per session kept for replay on (re)connect
`scrollback_lines`	`10000`	scrollback cap shown in the browser terminal
`font_size`	`16`	default terminal font size in pixels
`font_family`	`"Consolas, 'Cascadia Mono', monospace"`	terminal font stack
`theme`	a 16-color dark theme object	background, foreground, cursor, selection background, and the 16 ANSI colors
`shortcuts`	see the Shortcuts table below	interface keyboard shortcut map
`projects`	`[]`	launcher entries: `{"name", "path", "args": []}`
`snippets`	`[]`	snippet-bar entries: `{"name", "text", "send": false}`

Worked example: adding a project and a snippet

Edit `~/.claudiu/config.json` (or the file `CLAUDIU_CONFIG` points at) and add to the `projects` and `snippets` lists:

```
{
  "projects": [
    {"name": "My App", "path": "C:/code/my-app", "args": ["--continue"]}
  ],
  "snippets": [
    {"name": "run tests", "text": "run the test suite and fix any failures",
     "send": true}
  ]
}
```

Restart the server (or refresh the page after a `GET /api/config`-driven reload) to pick up the change. “My App” now appears in the project launcher, and “run tests” appears as a button in the snippets bar and in the `Ctrl+k` palette; because `send` is `true`, choosing it types the text and presses Enter.

Shortcuts

Interface-level keyboard shortcuts, all remappable via the `shortcuts` config key. They never intercept keys the CLI itself needs (plain `Ctrl+C`, `Ctrl+R`, arrow keys, and so on).

Action	Default key
`tab_1`	`Alt+1` — switch to tab 1
`tab_2`	`Alt+2` — switch to tab 2
`tab_3`	`Alt+3` — switch to tab 3
`tab_4`	`Alt+4` — switch to tab 4
`tab_5`	`Alt+5` — switch to tab 5
`tab_6`	`Alt+6` — switch to tab 6

Action	Default key
tab_7	Alt+7 — switch to tab 7
tab_8	Alt+8 — switch to tab 8
tab_9	Alt+9 — switch to tab 9
tab_prev	Alt+ArrowLeft — previous tab
tab_next	Alt+ArrowRight — next tab
new_session	Alt+t — open the new-session launcher
close_tab	Alt+w — close the active tab (asks; killing the session needs explicit confirmation)
font_bigger	Alt+= — increase this tab's font size
font_smaller	Alt+- — decrease this tab's font size
font_reset	Alt+0 — reset this tab's font size
search	Ctrl+Shift+f — search this tab's scrollback
snippet_palette	Ctrl+k — open the fuzzy-searchable snippet palette

Why Alt-based defaults: Chrome reserves Ctrl+T, Ctrl+W, Ctrl+Tab, and Ctrl+1…Ctrl+9 at the browser level — a web page cannot intercept them — so none of those combinations can ever be a working interface default; CLAUDIU's defaults use Alt instead. For the same reason, do not rebind any shortcut to bare Escape: the shortcut handler runs in the capture phase and would suppress the dialog-closing Escape handler that the launcher and rename dialogs rely on.

REST/WS API

Route	Description
GET /api/sessions	list live sessions
POST /api/sessions	create a session (body: path, args, title)
PATCH /api/sessions/<id>	rename a session (body: title)
DELETE /api/sessions/<id>	kill a session
GET /api/config	effective config plus load warnings
GET /api/resume	recent resumable Claude sessions
WS /ws/<id>	terminal stream (terminado protocol)

GET /api/resume scans the real ~/.claude/projects directory of the user running the server — the resume list you see is your own Claude Code history on that machine, in that account.

Command line

```
python -m claudiu [--config PATH] [--port PORT] [--log-dir DIR]
                  [--no-browser] [--verbose | --quiet] [--version]
```

Flag	Help
--config	path to config.json (default: ~/.claudiu/config.json, or $CLAUDIU_CONFIG)
--port	override the config port (config default: 8642)
--log-dir	directory for audit logs (default: <config dir>/logs)
--no-browser	do not open the browser automatically
--verbose	debug output on the console
--quiet	warnings and errors only on the console
--version	print the version and exit
--help	print the argument summary and exit

Logs

Every run writes an audit log to `<config dir>/logs/claudiu-<timestamp>.log` (override the directory with `--log-dir`) recording: the command line invoked, CLAUDIU/Python/Tornado/Terminado versions, any config load warnings, session start/rename/kill events, and how the run ended. Console verbosity is independent of the log file: `--verbose` prints debug lines to the console too, `--quiet` restricts the console to warnings and errors; the log file itself is always written at debug level.

Rebuilding this manual

This document is the source of truth; `USER_MANUAL.html` (and `USER_MANUAL.pdf`, when the tools are available) are built from it and committed so readers need no tooling of their own. To regenerate them after editing this file:

```
python docs/build_manual.py
```

Uses `pandoc` for the HTML and `pandoc` + `xelatex` for the PDF when both are on `PATH`; otherwise falls back to a small stdlib-only Markdown renderer for the HTML and prints that the PDF was skipped. Always exits 0. `--src` and `--outdir` override the input file and output directory.

Known limitations

- **Windows-first.** Development and hand-testing happen on Windows. Linux and macOS are exercised only by CI (unit, integration, and end-to-end tests all run there too) — they are not hand-tested.
- **One browser page per server.** The server assumes a single open page; it is not designed for multiple browser tabs/windows pointed at the same running instance.
- **No split panes.** One session per interface tab; no multi-pane layouts.
- **No remote access.** The server binds `127.0.0.1` only — it is not reachable from other devices, and there is no plan to make it so.
- **No transcript export.** CLAUDIU does not render or export session transcripts; that is the job of the separate claude-session-publisher tool.
- **`localStorage` use is minimal.** The only thing CLAUDIU stores in the browser's `localStorage` is each tab's remembered font size — nothing else about a session lives in the browser; the browser is a disposable view onto server-side sessions.
- **Keystrokes during a reconnect gap are dropped.** If you type while a tab's websocket is reconnecting (the "reconnecting…" strip is visible), those keystrokes are not queued and are lost — wait for the strip to clear before typing.